Chicago Housing Affordability Analysis

Core metrics

- **PIR** (Payment-to-Income Ratio): monthly mortgage payment / monthly household income
- **PTI** (Price-to-Income Ratio): anchored home price / annual household income

```
import os
import numpy as np
from pathlib import Path
import pandas as pd
import geopandas as gpd
import altair as alt
from sklearn.linear_model import LinearRegression
from shapely import wkt
import matplotlib.pyplot as plt

os.makedirs('output', exist_ok=True)
```

```
import warnings
warnings.filterwarnings('ignore')
alt.renderers.enable("png")
alt.data_transformers.disable_max_rows()

data_path = 'data/external'

# improve graph resolution
import tempfile
from IPython.display import SVG, display, Image
import vl_convert as vlc

def display_altair_png(chart, scale=2):
```

```
    """
    Render an Altair chart to a PNG and display it inline.

    Parameters
    ----------
    chart : altair.Chart
        Altair chart object to render.
    scale : int, optional
        Resolution scaling factor for the PNG (default = 2). Use scale=2 for standard slides
    """
    png_bytes = vlc.vegalite_to_png(chart.to_dict(), scale=scale)

    # Write to a temporary PNG file and display
    with tempfile.NamedTemporaryFile(suffix=".png") as tmp:
        tmp.write(png_bytes)
        tmp.flush()
        display(Image(filename=tmp.name))
```

```
::: {.cell execution_count=3}
``` {.python .cell-code}
# Data sources
urls = {
    'home_price': 'https://fred.stlouisfed.org/graph/fredgraph.csv?id=CHXRSA',
    'mortgage_rate': 'https://fred.stlouisfed.org/graph/fredgraph.csv?id=MORTGAGE30US',
    'income_median': 'https://fred.stlouisfed.org/graph/fredgraph.csv?id=MHIIL17000A052NCEN'
    'income_pc_real': 'https://fred.stlouisfed.org/graph/fredgraph.csv?id=RPIPC16980',
    'permits': 'https://fred.stlouisfed.org/graph/fredgraph.csv?id=CHIC917BPPRIVSA'
}

home = pd.read_csv(urls['home_price']).rename(columns={'observation_date':'date','CHXRSA':'ho
mort = pd.read_csv(urls['mortgage_rate']).rename(columns={'observation_date':'date','MORTGAGE
income_med = pd.read_csv(urls['income_median']).rename(columns={'observation_date':'date','MH
income_pc = pd.read_csv(urls['income_pc_real']).rename(columns={'observation_date':'date','RP
permits = pd.read_csv(urls['permits']).rename(columns={'observation_date':'date','CHIC917BPPR

for df in [home,mort,income_med,income_pc,permits]:
    df['date']=pd.to_datetime(df['date'])

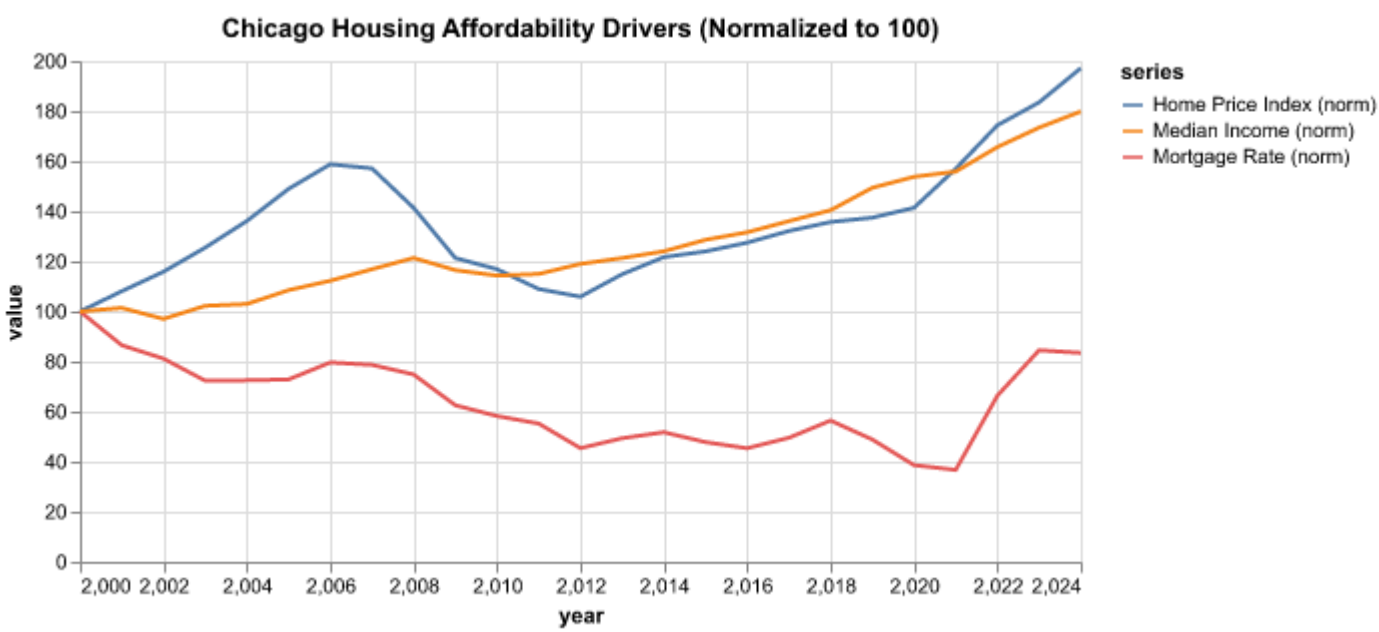

:::
```
```

```python
# Annualize and merge
start_date='2000-01-01'
home=home[home['date']>=start_date]
mort=mort[mort['date']>=start_date]
permits=permits[permits['date']>=start_date]
income_med=income_med[income_med['date']>=start_date]
income_pc=income_pc[income_pc['date']>=start_date]

home_y=home.set_index('date').resample('Y').mean().reset_index()
mort_y=mort.set_index('date').resample('Y').mean().reset_index()
permits_y=permits.set_index('date').resample('Y').mean().reset_index()

income_med['year']=income_med['date'].dt.year
income_pc['year']=income_pc['date'].dt.year
home_y['year']=home_y['date'].dt.year
mort_y['year']=mort_y['date'].dt.year
permits_y['year']=permits_y['date'].dt.year

merged=(home_y[['year','home_price_index']]
        .merge(mort_y[['year','mortgage_rate']],on='year',how='inner')
        .merge(income_med[['year','income_median']],on='year',how='inner')
        .merge(income_pc[['year','income_pc_real']],on='year',how='left')
        .merge(permits_y[['year','permits']],on='year',how='left')
        .dropna(subset=['home_price_index','mortgage_rate','income_median']))

merged=merged.sort_values('year').reset_index(drop=True)
```

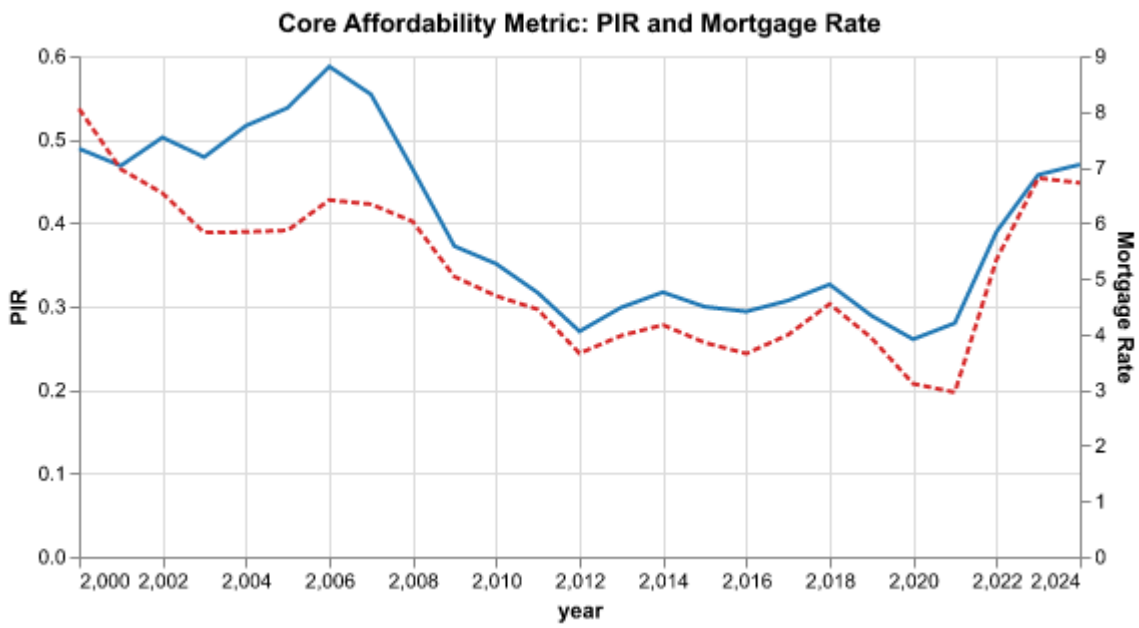
```python
# Build affordability metrics
# Anchored price proxy: choose baseline nominal price at first available year
P_BASE = 320_000
DOWN_PAYMENT = 0.20
N_MONTHS = 360

base_hpi = merged.loc[0,'home_price_index']
merged['home_price_proxy'] = P_BASE * (merged['home_price_index']/base_hpi)
merged['loan_amount'] = merged['home_price_proxy']*(1-DOWN_PAYMENT)
merged['r_month'] = (merged['mortgage_rate']/100)/12

# Mortgage payment formula
merged['mortgage_payment_monthly'] = merged['loan_amount'] * (
    merged['r_month']*(1+merged['r_month'])**N_MONTHS / ((1+merged['r_month'])**N_MONTHS - 1)
)
```

```
merged['income_monthly'] = merged['income_median']/12
# Payment-to-Income Ratio
merged['PIR'] = merged['mortgage_payment_monthly']/merged['income_monthly']
# Price-to-income Ratio
merged['PTI'] = merged['home_price_proxy']/merged['income_median']

# Normalized index
base_pir=merged.loc[0,'PIR']
merged['PIR_index']=merged['PIR']/base_pir*100
```

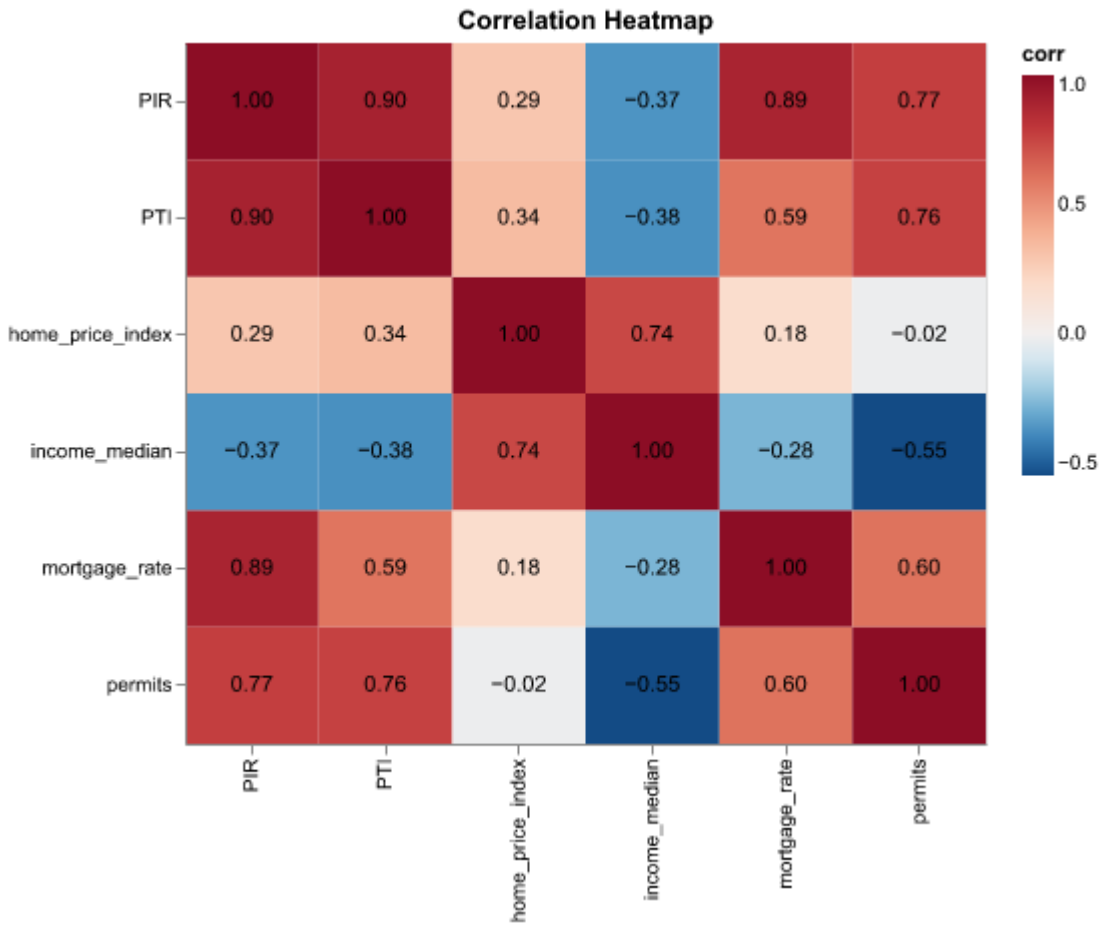
```
# Chart 1: trend overview (normalized)
plot_df=merged[['year','home_price_index','mortgage_rate','income_median']].copy()
for c in ['home_price_index','mortgage_rate','income_median']:
    plot_df[c+'_idx']=plot_df[c]/plot_df[c].iloc[0]*100

plot_long=plot_df.melt(id_vars=['year'],value_vars=['home_price_index_idx','mortgage_rate_idx
plot_long['series']=plot_long['series'].map({'home_price_index_idx':'Home Price Index (norm)
chart1=alt.Chart(plot_long).mark_line().encode(x='year',y='value',color='series').properties
chart1.save('output/trend_overview_normalized.png')
chart1
```



```
# Chart 2: PIR + PIR index
base=alt.Chart(merged)
left=base.mark_line(color='#1f77b4').encode(alt.X('year'),alt.Y('PIR',title='PIR'),color=alt
right=base.mark_line(strokeDash=[4,2],color='#d62728').encode(alt.X('year'),alt.Y('mortgage_r
chart2=alt.layer(left,right).resolve_scale(y='independent').properties(title='Core Affordabil
chart2.save('output/pir_main.png')
chart2
```



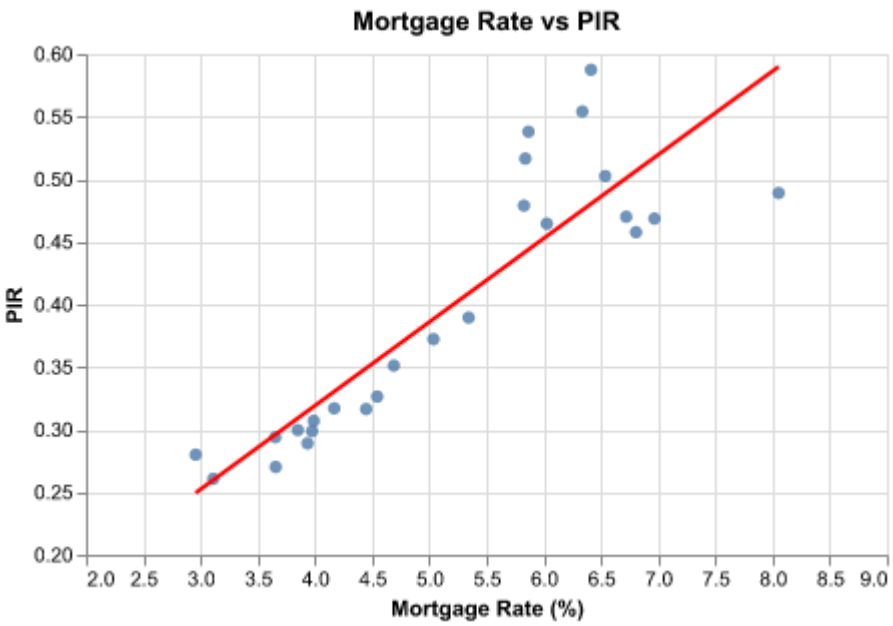

```
# Chart 3: Correlation heatmap
corr_cols=['home_price_index','mortgage_rate','income_median','permits','PIR','PTI']
corr=merged[corr_cols].corr(numeric_only=True)
corr_long=corr.reset_index().melt(id_vars='index').rename(columns={'index':'x','variable':'y
chart3=alt.Chart(corr_long).mark_rect().encode(x=alt.X('x',title=''),y=alt.Y('y',title=''),co
chart3=chart3+alt.Chart(corr_long).mark_text().encode(x='x',y='y',text=alt.Text('value',forma
chart3.save('output/correlation_heatmap.png')
chart3
```




```
# Chart 4: scatter with regression (mortgage rate vs PIR)
X=merged[['mortgage_rate']]
y=merged['PIR']
reg=LinearRegression().fit(X,y)
merged['pir_pred_by_rate']=reg.predict(X)

order=np.argsort(merged['mortgage_rate'].values)
line_df=merged.iloc[order][['mortgage_rate','pir_pred_by_rate']].copy()
merged5=merged[(merged['mortgage_rate']>=2)&(merged['mortgage_rate']<=9)]
line_df=line_df[(line_df['mortgage_rate']>=2)&(line_df['mortgage_rate']<=9)]
x_enc=alt.X('mortgage_rate',title='Mortgage Rate (%)',scale=alt.Scale(domain=[2,9]))
y_enc=alt.Y('PIR',title='PIR',scale=alt.Scale(domain=[0.2,0.6]))
scatter_chart=alt.Chart(merged5).mark_circle(size=40,opacity=0.8).encode(x_enc,y_enc)
line_chart=alt.Chart(line_df).mark_line(color='red').encode(x_enc,alt.Y('pir_pred_by_rate',s
chart5=alt.layer(scatter_chart,line_chart).properties(title='Mortgage Rate vs PIR',width=400
chart5.save('output/pir_vs_rate_regression.png')
```

```
chart5

print('Slope:', reg.coef_[0])
print('Intercept:', reg.intercept_)
print('R^2:', reg.score(X,y))
chart5
```

```
Slope: 0.06680891455265921
Intercept: 0.05189775888844317
R^2: 0.7853600801073992
```




```
# Save updated annual table
merged.to_csv('output/merged_annual_updated.csv', index=False)
merged.tail()
```

	year	home_price_index	mortgage_rate	income_median	income_pc_real	permits	home_p
20	2020	148.405477	3.111698	71243.0	60477.0	1256.116214	452470.4
21	2021	164.771937	2.957692	72215.0	62167.0	1529.794856	502369.8
22	2022	183.008395	5.344038	76744.0	59878.0	1433.657563	557970.5
23	2023	192.599684	6.806731	80346.0	62786.0	1181.536624	587213.2
24	2024	206.937817	6.721154	83327.0	NaN	1443.069769	630928.5

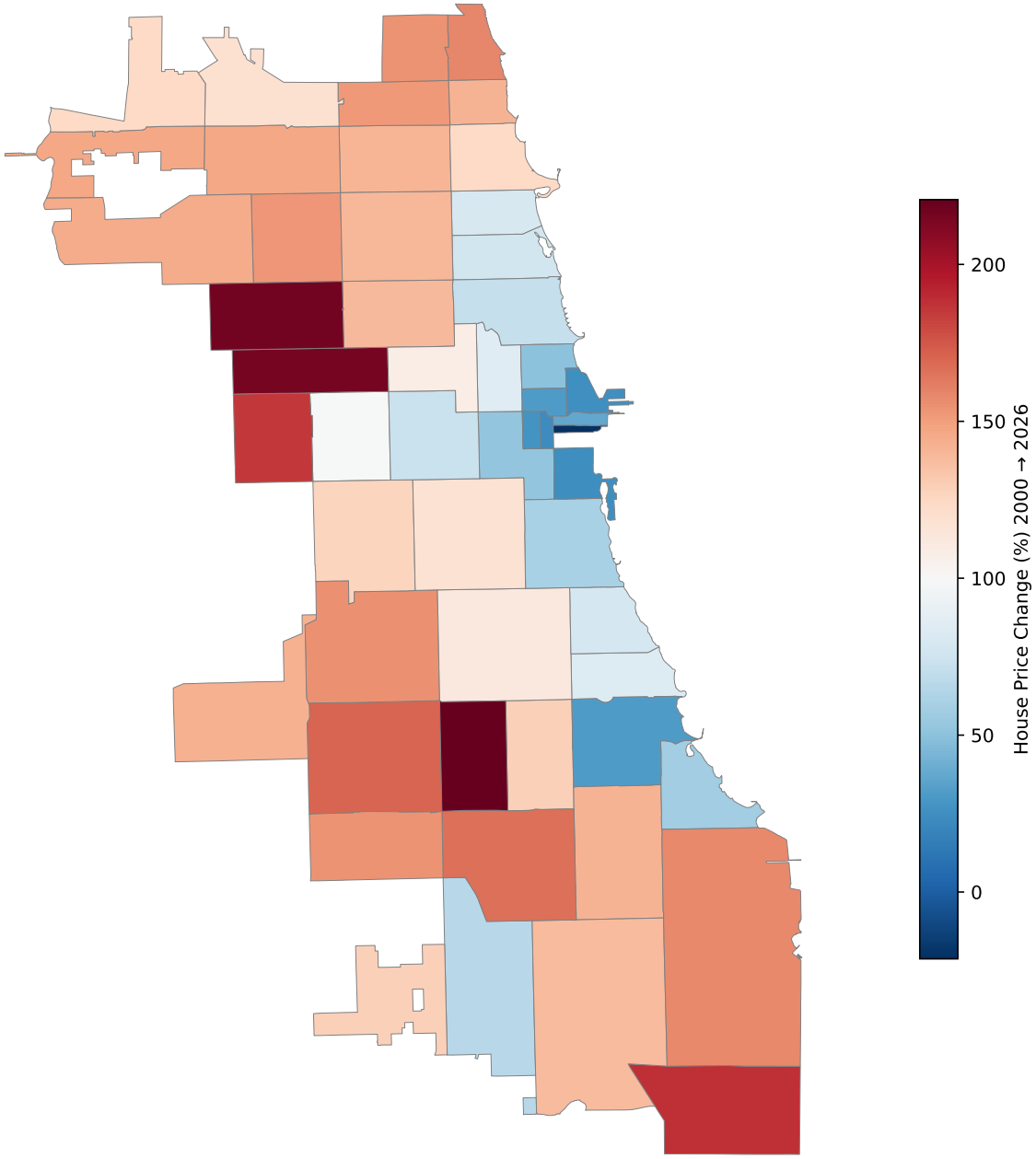
Spatial Graph

```
HERE = Path.cwd()
ROOT = HERE.parent
DATA_DIR = ROOT / "data" / "raw-data"
SAVE_DIR = ROOT / 'data' / 'output/'
df = pd.read_csv(DATA_DIR / "Zip_zhvi_uc_sfrcondo_tier_0.33_0.67_sm_sa_month.csv")
df2 = pd.read_csv(DATA_DIR / "Boundaries_-_ZIP_Codes_20260301.csv")
```

```
df_cook_house_price = df[
    (df['CountyName'] == 'Cook County') & (df['StateName'] == 'IL') & (df['City'] == 'Chicag
].copy()

print("df_cook_house_price.shape:", df_cook_house_price.shape)

print("df2.head():\n", df2.head())

zillow_zips = set(df_cook_house_price['RegionName'].astype(str).str.strip())
boundary_zips = set(df2['ZIP'].astype(str).str.strip())

common_zips = zillow_zips.intersection(boundary_zips)
only_in_zillow = zillow_zips - boundary_zips
only_in_boundary = boundary_zips - zillow_zips

print(f"Zillow Zip Code Number: {len(zillow_zips)}")
print(f"Map Zip Code Number: {len(boundary_zips)}")
print(f"Common Zip Code Number: {len(common_zips)}")
print("-" * 30)
print(f"Zip Code Not in Zillow: {only_in_boundary}")

df_cook_house_price['RegionName'] = df_cook_house_price['RegionName'].astype(str).str.strip(
date_cols = [c for c in df_cook_house_price.columns if '-' in str(c) and str(c)[:4].isdigit(

long = df_cook_house_price.melt(
    id_vars=['RegionName'],
    value_vars=date_cols,
    var_name='date_str',
    value_name='price'
)
long['year'] = long['date_str'].astype(str).str[:4].astype(int)
long = long.dropna(subset=['price'])
```

```
df_annual = long.groupby(['RegionName', 'year'], as_index=False)['price'].mean()
df_annual = df_annual.rename(columns={'RegionName': 'ZIP'})


year_min = 2000
year_max = df_annual['year'].max()

p_2000 = df_annual[df_annual['year'] == year_min].rename(columns={'price': 'price_2000'})[['Z
p_latest = df_annual[df_annual['year'] == year_max].rename(columns={'price': 'price_latest'}

change = p_2000.merge(p_latest, on='ZIP', how='inner')
change['pct_change'] = (change['price_latest'] - change['price_2000']) / change['price_2000']
change['pct_change_pct'] = change['pct_change'] * 100  #


df2['ZIP'] = df2['ZIP'].astype(str).str.strip()
geo = gpd.GeoDataFrame(df2, geometry=gpd.GeoSeries.from_wkt(df2['the_geom']), crs='EPSG:4326
merged = geo.merge(change, on='ZIP', how='inner')

fig, ax = plt.subplots(1, 1, figsize=(12, 10))
merged.plot(
    ax=ax,
    column='pct_change_pct',
    legend=True,
    cmap='RdBu_r',
    legend_kwds={'label': 'House Price Change (%) 2000 → ' + str(year_max), 'shrink': 0.6},
    edgecolor='gray',
    linewidth=0.3,
    missing_kwds={'color': 'lightgray', 'label': 'No Data'}
)
ax.set_title(f'Chicago (Cook County) Each ZIP House Price Change ({year_min}-{year_max})')
ax.axis('off')
plt.tight_layout()
plt.show()
```

```
df_cook_house_price.shape: (56, 322)
df2.head():
                                          the_geom  OBJECTID    ZIP  \
0  MULTIPOLYGON (((-87.64740454306485 41.88314614...         1  60607
1  MULTIPOLYGON (((-87.64081267674514 41.88907723...         2  60661
2  MULTIPOLYGON (((-87.63696184579966 41.88905665...         3  60606
3  MULTIPOLYGON (((-87.58103206721337 41.80421129...         4  60615
```

```
4  MULTIPOLYGON (((-87.66326342055945 41.72120432...         5  60643

        SHAPE_AREA       SHAPE_LEN
0  64,664,294.2344  39,143.6400726
1   9,357,755.8171  13,132.5659184
2  6,766,410.78996  12,040.4399607
3  66,565,454.6108  38,321.3132699
4  207,706,232.893  75,254.7301243
Zillow Zip Code Number: 56
Map Zip Code Number: 59
Common Zip Code Number: 56
------------------------------
Zip Code Not in Zillow: {'60666', '60827', '60707'}
```

Chicago (Cook County) Each ZIP House Price Change (2000–2026)